

Innovatech Solutions - Guia del Checklist

Preparacion de Videos | Parcial 3 | DuocUC 2026

Este documento analiza cada punto del checklist contra el estado real del proyecto. Para cada item se indica: estado actual (SI/PARCIAL/FALTA) y que decir o mostrar en el video.

VIDEO DE ARQUITECTURA (52 puntos)

I. Arquitectura de Componentes y Diseno Global

#1	Explica la problematica (caso seleccionado)?	SI
Que decir/mostrar: Explicar: Innovatech Solutions gestiona proyectos, tareas y equipos. El problema es que un sistema monolitico no escala cuando los modulos crecen de forma independiente.		
#2	Explica la descomposicion del sistema basandose en capacidades de negocio (DDD)?	SI
Que decir/mostrar: Mostrar diagrama: 3 subdominios (Projects, Tasks, Teams) = 3 microservicios + BFF como orquestador. Cada MS maneja su propia BD (Database per Service).		
#3	Describe cada uno de los componentes en la arquitectura?	SI
Que decir/mostrar: Listar: ms-projects (:3002), ms-tasks (:3001), ms-teams (:3003), bff-gateway (:3000), Traefik (:80), ms-monitor (:4000), frontend (:5173), 3 BD PostgreSQL.		
#4	Explica cada uno de los componentes en el diagrama?	SI
Que decir/mostrar: Mostrar docker-compose.yml y explicar el flujo: Frontend -> Traefik -> BFF (x3 replicas) -> MS -> BD. Explicar que Traefik balancea entre las 3 instancias del BFF.		

Service Discovery

#5	Explica Service Discovery y su funcion como base de datos dinamica de ubicaciones de red?	SI
Que decir/mostrar: Explicar: Docker DNS resuelve automaticamente los nombres de servicio (ms-tasks, ms-projects, etc.) a IPs internas. Traefik hace health checks para detectar instancias vivas.		
#6	Justifica la aplicacion de Service Discovery?	SI
Que decir/mostrar: Justificar: Para 4 servicios, Docker DNS + Traefik health checks es suficiente. Eureka/Consul seria overengineering para este contexto.		
#7	Describe como se trabaja el Service Discovery, lo bueno y lo malo?	SI
Que decir/mostrar: Bueno: zero-config, automatico con Docker. Malo: no tiene dashboard propio de discovery (Traefik lo compensa parcialmente), no soporta multi-cluster nativo.		
#8	Describe como se configura si requiere modificacion?	SI
Que decir/mostrar: Mostrar docker-compose.yml: los nombres de servicio son la configuracion. Si se agrega un MS nuevo, se agrega al compose y Docker DNS lo resuelve automaticamente.		
#9	Esta configurado mediante Self-Registration (ej. Eureka)?	PARCIAL
Que decir/mostrar: No usa Eureka. Usar Docker DNS como registro automatico: cada contenedor se auto-registra al iniciar. Traefik detecta instancias via health checks. Explicar la decision.		
#10	Muestra donde se encuentra ubicado para su descarga?	SI

	Que decir/mostrar: Mostrar: docker-compose.yml (Docker DNS es built-in, no se descarga), traefik/dynamic.yml (configuracion de health checks). Ambos en el repositorio.	
#11	Muestra como se levanta el servicio?	SI
	Que decir/mostrar: Ejecutar: docker compose up --build. Los servicios se registran automaticamente. Mostrar docker ps para ver los contenedores corriendo.	

Microservicios

#12	Explica el dominio que corresponde en todos los microservicios?	SI
	Que decir/mostrar: ms-projects: gestion de ciclo de vida de proyectos (PLANNED->IN_PROGRESS->COMPLETED). ms-tasks: tareas con asignacion a equipos. ms-teams: equipos con area y lider.	
#13	Explica las reglas de negocio en todos los microservicios?	SI
	Que decir/mostrar: Clave: No se puede completar un proyecto si tiene tareas pendientes (ms-projects consulta a ms-tasks). Factory valida campos obligatorios. Estados predefinidos por enum.	
#14	Explica el procesamiento de datos en todos los microservicios?	SI
	Que decir/mostrar: Flujo: Request -> Controller (HTTP) -> Service (logica + Prisma) -> Factory (validacion de creacion) -> PostgreSQL. Mostrar el codigo de cada capa.	
#15	Explica las validaciones de todos los microservicios?	SI
	Que decir/mostrar: Mostrar: ProjectFactory (nombre+desc obligatorios), TaskFactory (5 campos obligatorios), TeamFactory (nombre+desc+area+leaderId). Plus validaciones en update del Service.	
#16	Explica los casos de uso especificos?	SI
	Que decir/mostrar: CRUD completo en los 3 MS + updateStatus en projects y teams. Caso critico: completar proyecto verifica tareas pendientes via llamada inter-servicio.	
#17	Explica la estructura de carpetas y organizacion de clases?	SI
	Que decir/mostrar: Mostrar: src/controllers/, src/services/, src/factories/, src/middlewares/, prisma/schema.prisma, src/app.ts. Misma estructura en los 3 MS.	
#18	Explica las dependencias del microservicio?	SI
	Que decir/mostrar: Mostrar package.json: @prisma/client (ORM), express (HTTP), jsonwebtoken (JWT), axios (inter-service), swagger-jsdoc + swagger-ui-express (docs). Explicar cada una.	
#19	Explica los controladores?	SI
	Que decir/mostrar: Mostrar: Controller solo maneja HTTP (req/res), delega al Service. No tiene logica de negocio. Ejemplo: projectController.create llama a projectService.createProject.	
#20	Explica los metodos de seguridad?	SI
	Que decir/mostrar: JWT Zero Trust: cada MS valida el token independientemente con su propio middleware verifyToken. BFF autentica con bcrypt y genera JWT. El token se propaga en cada llamada.	
#21	Explica los patrones aplicados y donde en el codigo?	SI
	Que decir/mostrar: Factory: src/factories/ (creacion con validacion). Service Layer: src/services/ (logica separada). Circuit Breaker: bff/services/ (Opossum). Middleware Chain: src/middlewares/.	
#22	Explica los archivos de configuracion?	SI
	Que decir/mostrar: Mostrar: .env (DATABASE_URL, JWT_SECRET), docker-compose.yml (orquestracion), prisma/schema.prisma (modelos), tsconfig.json (TypeScript), jwt.config.ts (secreto JWT).	

#23	Explica como levantar el microservicio?	SI
	Que decir/mostrar: docker compose up --build. Individual: cd ms-projects && npm run dev. Mostrar los logs de cada servicio iniciando y confirmando puerto.	
#24	Explica como maneja las excepciones?	SI
	Que decir/mostrar: try/catch en Controller retorna HTTP 400/404/500 con mensaje descriptivo. Factory lanza Error con mensaje especifico. Service atrapa errores de Prisma.	
#25	Explica buenas practicas en la creacion de un microservicio?	SI
	Que decir/mostrar: Separacion de capas (Controller-Service-Factory), Database per Service, tipado con TypeScript, version fija de Prisma (5.22.0 sin ^), Conventional Commits.	
#26	Explica como se configura Circuit Breaker?	SI
	Que decir/mostrar: Mostrar bff/services/: Opossum con timeout=3s, errorThreshold=50%, resetTimeout=10s. Fallbacks: array vacio para getAll, mensaje de error para operaciones especificas.	
#27	Explica buenas practicas HttpStatus 201, 500, etc.?	SI
	Que decir/mostrar: Mostrar: 201 para POST exitoso, 200 para GET/PATCH/DELETE, 400 para validacion, 401 sin token, 403 token invalido, 404 no encontrado, 500 error interno.	
#28	Tiene log interno para seguimiento y mejoras?	PARCIAL
	Que decir/mostrar: console.log/console.error basico. No hay Winston ni logging estructurado. Mencionar que como mejora futura se implementaria Winston con niveles de log.	
#29	Tiene metricas internas cada microservicio?	PARCIAL
	Que decir/mostrar: ms-monitor consulta /health de cada servicio (uptime, latencia). No hay Prometheus/Grafana. Mencionar como mejora futura para el dashboard de metricas.	
#30	Informa pruebas unitarias, integracion, frontend?	SI
	Que decir/mostrar: Mostrar: 82 tests pasando. Jest + Supertest. Unitarias (factories, services con mock Prisma), Integracion (auth routes), E2E (flujo completo de negocio con Circuit Breaker).	

Seguridad

#31	Justifica la aplicacion de seguridad?	SI
	Que decir/mostrar: Zero Trust: cada servicio valida JWT independientemente. Si el BFF es comprometido, los MS internos siguen protegidos. Bcrypt para passwords (no texto plano).	
#32	Explica como se genera el JWT?	SI
	Que decir/mostrar: Mostrar auth.routes.ts: POST /auth/login -> bcrypt.compare(password, hash) -> jwt.sign({username, role, service}, secret, {expiresIn: 24h}) -> retorna token.	
#33	Explica como se configura el JWT?	SI
	Que decir/mostrar: Mostrar jwt.config.ts: secret desde JWT_SECRET env var con fallback. ExpiresIn: 24h. Payload: username, displayName, role, service. Algoritmo: HS256 (default).	
#34	Explica como se valida el JWT?	SI
	Que decir/mostrar: Mostrar auth.middleware.ts: extrae token del header Authorization, jwt.verify(token, secret), si falla retorna 403. Tolerante a Bearer/bearer/token directo.	
#35	Explica como se enruta la peticion aplicando seguridad?	SI

Que decir/mostrar: Mostrar app.ts del BFF: `app.use("/api", verifyToken, [routers])`. Swagger y `/health` son publicos. `/auth/login` es publico. Todo `/api/*` requiere JWT valido.

API Gateway

#36	Explica el funcionamiento de API Gateway?	SI
	Que decir/mostrar: BFF como gateway: punto de entrada unico en :3000. Traefik en :80 balancea entre replicas del BFF. El frontend solo habla con Traefik, nunca con los MS directamente.	
#37	Explica los componentes que integran API Gateway?	SI
	Que decir/mostrar: Mostrar: <code>auth.routes.ts</code> (login), <code>middleware/auth.middleware.ts</code> (JWT), <code>routes/</code> (proxy a MS), <code>services/</code> (Circuit Breaker + axios), <code>config/</code> (JWT, Swagger).	
#38	Explica como filtra las peticiones?	SI
	Que decir/mostrar: Middleware chain: 1) CORS (origenes permitidos), 2) <code>express.json</code> (parsea body), 3) <code>verifyToken</code> (JWT), 4) route handler. Si falla en cualquier paso, retorna error.	

Monitoreo de Sistema

#39	Justifica la aplicacion de monitoreo?	SI
	Que decir/mostrar: Sin monitoreo no sabemos si un MS cayo hasta que un usuario reporta el error. <code>ms-monitor</code> detecta caidas en 15 segundos y muestra el estado en un dashboard en tiempo real.	
#40	Explica las acciones a realizar si un microservicio falla?	SI
	Que decir/mostrar: Circuit Breaker retorna fallback inmediato (no bloquea). Docker restart: always reintenta levantar el contenedor. Swarm: auto-healing con replicas. Monitor alerta visualmente.	
#41	Explica como se entera si un microservicio falla?	SI
	Que decir/mostrar: Mostrar <code>ms-monitor</code> : consulta GET <code>/health</code> cada 15s. Dashboard HTML muestra UP/DOWN con latencia. Endpoint <code>/status</code> retorna JSON para integracion. Traefik health checks.	
#42	Tiene seguimiento para mejoras en el monitoreo?	PARCIAL
	Que decir/mostrar: Actualmente basico (health checks). Mejora futura: Prometheus + Grafana para metricas de CPU/memoria/requests, alertas por email/Slack, historico de disponibilidad.	

Frontend

#43	Explica la funcionalidad y por que fue elegida la tecnologia?	SI
	Que decir/mostrar: React + Vite + TypeScript. React: componentes reutilizables, ecosistema amplio. Vite: build rapido, HMR instantaneo. TypeScript: tipado que previene errores.	
#44	Explica las reglas de negocio en el frontend?	SI
	Que decir/mostrar: Login obligatorio para acceder. Token se almacena en <code>localStorage</code> . Tabs para navegar Projects/Tasks/Teams. CRUD completo desde la interfaz.	
#45	Explica buenas practicas, ej: trabaja con TypeScript?	SI
	Que decir/mostrar: TypeScript en todo el frontend (.tsx). Componentes separados por vista (<code>ProjectsView</code> , <code>TasksView</code> , <code>TeamsView</code>). Cliente HTTP centralizado en <code>bffClient.ts</code> con interceptor JWT.	
#46	Explica la seguridad aplicada?	SI

	Que decir/mostrar: Mostrar bffClient.ts: interceptor agrega Bearer token a cada request. Login almacena token en localStorage. Logout limpia token. Sin token -> redirige a login.	
#47	Explica acciones si una petición no recibe respuesta?	PARCIAL
	Que decir/mostrar: El Circuit Breaker en el BFF retorna fallback. El frontend muestra el error del catch en la UI. Mejora futura: retry automatico, loading spinners, mensajes amigables.	
#48	Explica como trabaja con los tiempos de respuesta?	PARCIAL
	Que decir/mostrar: Circuit Breaker timeout: 3s. Si el MS no responde en 3s, fallback inmediato. Mejora futura: indicadores de carga en frontend, timeout configurable por operacion.	
#49	Define estandares de rendimiento y disponibilidad?	PARCIAL
	Que decir/mostrar: Disponibilidad: Docker restart:always + Swarm auto-healing. Rendimiento: Circuit Breaker evita degradacion en cascada. Mejora futura: SLA definido, metricas de latencia p95.	
#50	Tiene log interno en el frontend?	PARCIAL
	Que decir/mostrar: console.error en catches. No hay logging estructurado en frontend. Mejora futura: servicio de logging que envíe errores del frontend a un endpoint centralizado.	
#51	Explica manejo de mensajes concordantes con el negocio ante excepciones?	SI
	Que decir/mostrar: Los MS retornan mensajes descriptivos ("El nombre es obligatorio", "tareas pendientes"). El frontend muestra estos mensajes directamente al usuario.	
#52	Explica el manejo de excepciones?	SI
	Que decir/mostrar: try/catch en cada llamada API del frontend. Error se muestra en la UI. Si el BFF no responde, Circuit Breaker retorna fallback y el frontend muestra lista vacía o mensaje.	

VIDEO DE USO (9 puntos)

Video de Uso

#1	Explica la problematica y como entrega solucion?	SI
Que decir/mostrar: Problema: gestionar proyectos, tareas y equipos de forma escalable. Solucion: sistema de microservicios donde cada dominio es independiente, con BFF como punto unico.		
#2	Realiza una introduccion de la solucion?	SI
Que decir/mostrar: Innovatech Solutions: plataforma de gestion con arquitectura de microservicios, 4 servicios backend, frontend React, load balancer Traefik, monitoreo en tiempo real.		
#3	Indica cuales son los requisitos del sistema?	SI
Que decir/mostrar: Docker Desktop, Node.js v22+, Git. Opcional: VS Code, Postman/Thunder Client. Sistema operativo: Windows/Mac/Linux con Docker.		
#4	Explica la instalacion y configuracion?	SI
Que decir/mostrar: git clone -> docker compose up --build. Explicar que un solo comando levanta los 8 contenedores. Mostrar los logs iniciando.		
#5	Indica como acceder al sistema?	SI
Que decir/mostrar: Frontend: http://localhost:5173. API BFF via Traefik: http://localhost. Swagger: http://localhost:3000/api-docs. Monitor: http://localhost:4000. Credenciales: admin/admin123.		
#6	Realiza una descripcion de la interfaz?	SI
Que decir/mostrar: Pantalla de login -> Dashboard con tabs (Projects, Tasks, Teams) -> CRUD en cada tab -> Logout. Mostrar navegacion completa.		
#7	Indica las funcionalidades principales?	SI
Que decir/mostrar: Login con bcrypt, CRUD proyectos/tareas/equipos, regla de negocio de tareas pendientes, balanceo de carga, monitoreo de servicios, Circuit Breaker.		
#8	Explica toda la funcionalidad del sistema?	SI
Que decir/mostrar: Demo completa: login -> crear proyecto -> crear tarea -> asignar a equipo -> completar tarea -> completar proyecto. Mostrar Swagger, Monitor, Traefik dashboard.		
#9	Entrega conclusion y posibilidad de escalar?	SI
Que decir/mostrar: Escalabilidad: agregar replicas con --scale, Docker Swarm para produccion, agregar nuevos MS sin afectar los existentes. Mejoras: Prometheus, logging centralizado, CI/CD.		

RESUMEN

Estado	Cantidad	Porcentaje
SI - Cubierto	53	86%
PARCIAL - Requiere mencion	8	13%
FALTA - No implementado	0	0%
TOTAL	61	100%

